

Cloud Security Essentials TASK -1

- 1. Write a short note explaining the difference between symmetric and asymmetric encryption, and give one real-world example of where each might be used in a cloud app like WhatsApp or Paytm.**

Ans :

Symmetric Encryption

- Uses one secret key for both encryption and decryption.
- It is fast and efficient for encrypting large amounts of data.
- The sender and receiver must both have the same secret key.

Asymmetric Encryption

- Uses two keys: a public key for encryption and a private key for decryption.
- It is more secure for sharing keys and authentication but is slower than symmetric encryption.
- The public key can be shared openly, while the private key is kept secret.

- 2. Compare server-side encryption and client-side encryption by listing 3 pros and 3 cons for each method in the context of storing user photos on a cloud platform like Instagram.**

Ans :

Server-Side Encryption (SSE)

- **Definition:**
The cloud provider encrypts the photos after they are uploaded and decrypts them when an authorized user accesses them.

Pros

- Easy to use – Users do not need to manage encryption keys.

- Automatic encryption – All uploaded photos are protected without extra effort.
- Supports cloud features – Search, backup, sharing, and photo processing work easily.

Cons

- Cloud provider can access data if it has the encryption keys.
- Less privacy because users must trust the cloud provider.
- Risk if the server is compromised, as attackers may gain access to encrypted data and keys.

Client-Side Encryption (CSE)

- Definition:
The user's device encrypts photos before uploading them to the cloud. The cloud stores only encrypted data.

Pros

- Better privacy – The cloud provider cannot view user photos.
- Greater security – Data remains encrypted even if the cloud server is breached.
- User controls the encryption keys, reducing the risk of unauthorized access.

Cons

- Key management is the user's responsibility; losing the key may mean losing access to the photos.
- More complex to implement than server-side encryption.
- Some cloud features (such as searching or automatic photo analysis) may be limited or unavailable.

3. Set up an AWS S3 bucket (or use any cloud storage emulator like MinIO if you don't have an AWS account) and enable server-side encryption using AES-256 for all uploaded files. Upload a sample file and verify encryption is active.

Ans :

- Log in to the AWS Management Console.
- Open the Amazon S3 service.
- Click Create bucket.
- Enter a unique bucket name (e.g., my-secure-bucket).
- Select your preferred AWS Region.
- Keep the default settings (or modify them if required).
- Click Create bucket.
- Open the newly created bucket.
- Go to the Properties tab.
- Scroll down to the Default encryption section.
- Click Edit.
- Enable Server-side encryption.
- Select SSE-S3 (Amazon S3 managed keys).
- Ensure the encryption algorithm is AES-256.
- Click Save changes.
- Go to the Objects tab.
- Click Upload.
- Select a sample file (e.g., sample.txt).

- Click Upload to upload the file.
- After the upload is complete, click the uploaded file.
- Open the Properties tab of the file.
- Scroll to the Server-side encryption section.
- Verify that the encryption status shows:
 - Encryption Type: SSE-S3
 - Algorithm: AES-256
 - Status: Enabled

**4. Simulate uploading a playlist cover image to your 'Spotify-style' S3 bucket with server-side encryption enabled, then use the AWS CLI or MinIO client to confirm the encryption status of the uploaded object.

Hint: Use the 'aws s3api head-object' command or MinIO's object info command to check encryption metadata.**

Ans :

- Ensure your S3 bucket has Server-Side Encryption (SSE-S3 with AES-256) enabled.
- Prepare a sample playlist cover image (e.g., playlist-cover.jpg).
- Upload the image to your S3 bucket using the AWS CLI:
 - `aws s3 cp playlist-cover.jpg s3://spotify-style-bucket/`
 - After the upload completes, verify the encryption status using
 - `aws s3api head-object --bucket spotify-style-bucket --key playlist-cover.jpg`
 - In the command output, look for the following field
 - "ServerSideEncryption": "AES256"

- If "ServerSideEncryption": "AES256" is displayed, the uploaded image is encrypted using Server-Side Encryption (AES-256).

5. Use ChatGPT or Copilot to generate a code snippet in Python or Node.js that uploads a file to an S3 bucket with server-side encryption enabled. Test the code with your own file and explain how the encryption parameter works.

Ans :

```
import boto3
```

```
# Create an S3 client
```

```
s3 = boto3.client('s3')
```

```
# Upload a file with Server-Side Encryption (AES-256)
```

```
s3.upload_file(  
    Filename='sample.txt',  
    Bucket='spotify-style-bucket',  
    Key='sample.txt',  
    ExtraArgs={  
        'ServerSideEncryption': 'AES256'  
    }  
)
```

```
print("File uploaded successfully with Server-Side Encryption  
(AES-256).")
```

- Install the AWS SDK for Python
- `pip install boto3`
- Configure your AWS credentials
- `aws configure`
- Enter your:
 - AWS Access Key ID
 - AWS Secret Access Key
 - Region
 - Output format
- Save the Python code as `upload.py`.
- Place a sample file (e.g., `sample.txt`) in the same folder.
- Replace `spotify-style-bucket` with your actual S3 bucket name.
- Run the program
- `python upload.py`
- Verify encryption
- `aws s3api head-object --bucket spotify-style-bucket --key sample.txt`